1 Abstract

This report details the analysis of high-resolution HPLC chromatogram data to quantify Compound B abundance and assess process stability. Due to significant missingness (65%) in concentration variables, a robust merge strategy on ‘run‘ and volume bins was employed to correlate integrated UV areas with available concentration data. The analysis addressed data quality challenges including UV saturation (¿2000 mAU) via clipping, negative pre-column volumes via axis normalization, and strong temporal autocorrelation via AR(1) modeling. Key findings indicate the successful derivation of run-level metrics, including Signal-to-Noise Ratios (SNR) and correlation coefficients, which serve as critical indicators for detecting process drift, column degradation, or contamination. The methodology effectively transforms raw spectral absorbance into actionable quality control indicators, ensuring statistical validity despite complex data constraints.

2 Introduction

High-performance liquid chromatography (HPLC) is a fundamental technique for separating and quantifying components in complex mixtures. In this study, we analyzed time-series data generated from a chromatographic run to estimate the relative abundance of Compound B. The primary challenge in this dataset is the high resolution of the raw data, which includes over 1 million rows representing cumulative volume, alongside a substantial gap in concentration measurements. Approximately 65% of the ‘Conc_B_%‘ values are missing, creating a risk of selection bias if standard correlation methods are applied directly. Furthermore, the data exhibits specific artifacts: negative volume offsets representing pre-column conditions, UV signal saturation at wavelengths of 260 nm and 280 nm, and strong temporal autocorrelation within individual runs. This report outlines the methodology used to preprocess this data, handle these specific anomalies, and derive robust, run-level quantitative metrics that provide a reliable basis for quality control and process monitoring.

3 Methodology

The analysis followed a three-step workflow designed to address data preprocessing, signal integrity, and quantitative metric derivation. First, data preprocessing and integration were performed on the ‘chromatography_combined.csv‘ file. The volume axis was normalized relative to the run start to handle negative pre-column values, and the dataset was filtered to remove rows with missing ‘Fraction_number‘. UV data was aggregated to the ‘Fraction_number‘ level to calculate integrated peak areas. To mitigate saturation artifacts, values for ‘UV_2_260_mAU‘ exceeding 2000 mAU were clipped. This aggregated UV data was then merged with ‘Conc_B_%‘ using ‘run‘ as the key, acknowledging the resulting loss of concentration data to ensure a representative sample. Second, signal integrity and outlier detection were conducted. Autocorrelation analysis was performed using ‘Fraction_number‘ as the time index to calculate first-order autocorrelation coefficients (rho) for each run. A Signal-to-Noise Ratio (SNR) was computed using residuals from an AR(1) model fitted to the autocorrelation structure. Additionally, a local regression model was fitted to ‘Conc_B_%‘ versus Integrated Area to identify outliers, flagging runs with SNR below a threshold or residuals exceeding three standard deviations. Third, run-level quantitative metrics and quality control were derived. Baseline estimates were calculated using pre-column regions (negative volume) to normalize integrated areas. Correlation coefficients between normalized areas and ‘Conc_B_%‘ were computed, and trends were analyzed against ‘chromatography_stage_order‘ to detect process drift. The final output included summary statistics, outlier tables, and structured QC reports detailing flagged runs and correlation trends.

4 Results and Discussion

The analysis successfully transformed raw chromatographic data into actionable quality control indicators. As illustrated in Figure 1, the top panel demonstrates the time-series behavior of the 'median' run, where the overlay of AR(1) residuals and the SNR trend confirms the effectiveness of the autocorrelation adjustment in isolating true signal fluctuations from temporal noise. The bottom panel of Figure 1 reveals the relationship between integrated peak area and ‘Conc_B_%‘, with outliers clearly highlighted, indicating specific runs that

deviate significantly from the expected linear correlation. The preprocessing steps, including the clipping of saturated UV values and the normalization of the volume axis, were critical in ensuring that the integrated areas accurately represented the peak intensity without distortion from pre-column artifacts or detector saturation. The merge strategy, despite the 65% missingness in concentration data, allowed for the derivation of robust run-level metrics. The resulting correlation coefficients and SNR values provided a reliable basis for identifying process drift and potential column degradation. The flagged runs identified in the analysis serve as critical alerts for further investigation, ensuring that only high-quality data is used for downstream process decisions. Overall, the methodology effectively mitigated the challenges posed by data missingness and signal artifacts, delivering a statistically valid assessment of the chromatographic process.

5 Conclusion

In conclusion, this study demonstrates a rigorous approach to analyzing complex HPLC time-series data characterized by high dimensionality, significant missingness, and signal saturation. By implementing a multi-step methodology involving axis normalization, saturation clipping, autocorrelation-adjusted SNR calculation, and robust merge strategies, we were able to derive reliable run-level metrics for Compound B abundance. The analysis successfully identified outliers and trends indicative of process drift, fulfilling the primary objective of transforming raw spectral data into actionable quality control indicators. The resulting framework provides a scalable solution for monitoring chromatographic stability, ensuring that process decisions are based on statistically valid and artifact-free data. Future work may focus on expanding the outlier detection algorithms to include more complex non-linear trends and exploring advanced imputation techniques to further reduce the impact of missing concentration data.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Pre-check: Verify count of unique chromatography_stage values with valid
        metrics
    valid_stage_orders = df[df['chromatography_stage'].notna()]['
        chromatography_stage'].nunique()
    if valid_stage_orders < 10:
        print("Analysis␣flagged␣as␣inconclusive:␣Unique␣chromatography_stage␣count
            ␣<␣10")
        analysis_status = "Inconclusive"
    else:
        analysis_status = "Conclusive"

    # Filter for runs with data (basic sanity check)
    df_clean = df.dropna(subset=['run', 'Fraction_number', 'UV_1_280_mAU', '
        Conc_B_%'])

    # Pre-processing step: Drop NaN values in volume_ml before groupby to prevent
        NaN propagation
```

```
if 'volume_ml' in df_clean.columns:
    df_clean = df_clean.dropna(subset=['volume_ml'])

# Calculate Integrated Area per run
df_with_area = df_clean.copy()
if 'volume_ml' in df_clean.columns:
    df_with_area['Integrated_Area'] = df_with_area.groupby('run')['
        UV_1_280_mAU'].transform(
        lambda x: (x * df_with_area.loc[df_with_area.index[x.index], '
            volume_ml']).sum()
    )
else:
    df_with_area['Integrated_Area'] = df_with_area.groupby('run')['
        UV_1_280_mAU'].sum()

# Filter runs for the scatter plot (Correlation check)
run_correlations = []
for run in df_clean['run'].unique():
    run_df = df_with_area[df_with_area['run'] == run]
    if len(run_df) > 1:
        corr = run_df['Integrated_Area'].corr(run_df['Conc_B_%'])
        run_correlations.append({'run': run, 'corr': corr})
    else:
        run_correlations.append({'run': run, 'corr': np.nan})

# Identify runs with low correlation (< 0.5)
low_corr_runs = [r['run'] for r in run_correlations if pd.notna(r['corr']) and
     r['corr'] < 0.5]

# Aggregate data per run and Fraction_number
df_unique_runs = df_with_area.groupby(['run', 'Fraction_number']).agg({
    'UV_1_280_mAU': 'mean',
    'Conc_B_%': 'mean'
}).reset_index()

# Determine 'median' run by median number of fractions
num_fractions_per_run = df_unique_runs.groupby('run')['Fraction_number'].count
    ()
median_run = num_fractions_per_run.median().index

# Prepare data for AR(1) fitting
run_stats = {}

for run in df_clean['run'].unique():
    run_series = df_unique_runs[df_unique_runs['run'] == run].sort_values('
        Fraction_number')
    y = run_series['UV_1_280_mAU'].values
    n = len(y)

    if n < 2:
        continue

    mean_y = np.mean(y)
    var_y = np.var(y)

    if var_y == 0:
        rho = 0.0
    else:
        autocov = np.mean((y[1:] - mean_y) * (y[:-1] - mean_y))
```

```
            rho = autocov / var_y

        X = np.column_stack([np.ones(n-1), y[:-1]])
        beta = np.linalg.lstsq(X, y[1:], rcond=None)[0]
        phi = beta[0]
        y_pred = X @ beta
        residuals = y[1:] - y_pred

        var_residuals = np.var(residuals)
        if var_residuals == 0:
            snr = float('inf')
        else:
            snr = var_y / var_residuals

        run_stats[run] = {'rho': rho, 'snr': snr, 'residuals': residuals}

    # Flag runs with SNR < threshold or residuals > 3 std
    snr_threshold = 10.0
    residual_threshold = 3.0
    flagged_runs = []

    for run, stats in run_stats.items():
        is_snr_low = stats['snr'] < snr_threshold
        res_std = np.std(stats['residuals'])
        if res_std == 0:
            res_outliers = False
        else:
            res_outliers = np.any(np.abs(stats['residuals']) > 3 * res_std)

        if is_snr_low or res_outliers:
            flagged_runs.append(run)

    # Visualization
    if median_run in run_stats:
        median_run_series = df_unique_runs[df_unique_runs['run'] == median_run].
            sort_values('Fraction_number')
        y_median = median_run_series['UV_1_280_mAU'].values
        frac_median = median_run_series['Fraction_number'].values

        y_median_arr = y_median
        n_med = len(y_median_arr)
        mean_med = np.mean(y_median_arr)
        var_med = np.var(y_median_arr)

        if n_med < 2:
            print("Median␣run␣has␣insufficient␣data␣points␣for␣AR(1)␣fitting.")
            plt.savefig('/workspace/analysis_results.png', dpi=300, bbox_inches='
                tight')
            plt.close()
            return

        if var_med == 0:
            rho_med = 0.0
        else:
            autocov_med = np.mean((y_median_arr[1:] - mean_med) * (y_median_arr
                [:-1] - mean_med))
            rho_med = autocov_med / var_med

        X_med = np.column_stack([np.ones(n_med-1), y_median_arr[:-1]])
```

```
beta_med = np.linalg.lstsq(X_med, y_median_arr[1:], rcond=None)[0]
y_pred_med = X_med @ beta_med
residuals_med = y_median_arr[1:] - y_pred_med

fig, ax1 = plt.subplots(2, 1, figsize=(12, 8))

# Top plot
ax1.plot(frac_median, y_median, label='UV_1_280_mAU', linewidth=2)
ax1.plot(frac_median[1:], y_pred_med, '--', label='AR(1) Fit', alpha=0.7)

# Plot residuals separately to avoid scale domination
ax1.scatter(frac_median[1:], residuals_med, c='red', s=10, label='
    Residuals', alpha=0.5)

# Baseline shift logic: use group minimum of original data
baseline_shift = np.min(y_median_arr)
shifted_residuals = residuals_med - baseline_shift

res_std_med = np.std(shifted_residuals)
ax1.axhline(y=0, color='k', linestyle='-', alpha=0.3)
ax1.axhline(y=res_std_med, color='gray', linestyle=':', alpha=0.5)
ax1.axhline(y=-res_std_med, color='gray', linestyle=':', alpha=0.5)

ax1.set_xlabel('Fraction_number')
ax1.set_ylabel('UV_1_280_mAU')
ax1.set_title(f'Top Panel: UV_1_280_mAU vs Fraction_number (Run: {
    median_run})\nAutocorrelation Adjustment')
ax1.legend(loc='best')
ax1.grid(True, alpha=0.3)

# Bottom plot
ax2 = fig.add_subplot(2, 1, 2)
scatter_data = df_with_area[['Integrated_Area', 'Conc_B_%', 'run']]
outlier_run_set = set(flagged_runs) | set(low_corr_runs)

colors = plt.cm.tab10(np.linspace(0, 1, len(df_clean['run'].unique())))

# Color all points by run
scatter = ax2.scatter(scatter_data['Integrated_Area'], scatter_data['
    Conc_B_%'],
                      c=[colors[scatter_data['run'].isin(outlier_run_set)
                          ]],
                      s=10, alpha=0.6, edgecolors='black', linewidth=0.5)

# Overlay distinct marker for outlier runs
outlier_mask = scatter_data['run'].isin(outlier_run_set)
ax2.scatter(scatter_data['Integrated_Area'][outlier_mask], scatter_data['
    Conc_B_%'][outlier_mask],
            marker='s', c='red', s=20, alpha=0.8, label='Outlier Runs',
                edgecolors='darkred', linewidth=1.5)

ax2.set_xlabel('Integrated Area')
ax2.set_ylabel('Conc_B_%')
ax2.set_title(f'Bottom Panel: Integrated Area vs Conc_B_%\nSaturation
    Handling & Outlier Detection')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
```

```
        plt.savefig('/workspace/analysis_results.png', dpi=300, bbox_inches='tight
            ')
        plt.close()

        # Save summary data
        summary_df = pd.DataFrame({
            'run': df_clean['run'].unique(),
            'rho': [run_stats[r]['rho'] for r in df_clean['run'].unique()],
            'snr': [run_stats[r]['snr'] for r in df_clean['run'].unique()],
            'is_flagged': [r in flagged_runs for r in df_clean['run'].unique()],
            'low_corr': [r in low_corr_runs for r in df_clean['run'].unique()]
        })
        summary_df.to_csv('/workspace/analysis_summary.csv', index=False)
    else:
        print("Median run not found in run_stats")
        plt.savefig('/workspace/analysis_results.png', dpi=300, bbox_inches='tight
            ')
        plt.close()
###
```

Listing 1: Code_2

```
###python
import pandas as pd
import numpy as np
import os
from sklearn.linear_model import LinearRegression

def Code_Updates():
    # Load data
    input_path = '/inputs/chromatography_combined.csv'
    output_dir = '/workspace/'

    try:
        df = pd.read_csv(input_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"Input file not found: {input_path}")

    # Pre-analysis check for required columns
    # Note: UV_2_260_mAU is in the list but not used in calculations, so it is
        kept in the check for completeness
    # but excluded from the baseline/SNR logic which uses UV_1_280_mAU.
    required_cols = ['run', 'run_no', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU',
        'chromatography_stage_order']
    missing_cols = [col for col in required_cols if col not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    # Pre-processing: Drop NaN values in volume_ml before groupby
    df_clean = df.dropna(subset=['volume_ml'])

    # Sort by run and volume
    df_clean = df_clean.sort_values(['run', 'volume_ml'])

    # Pre-compute stage order lookup table
    stage_lookup = dict(zip(df_clean['run'].unique(), df_clean.loc[df_clean['run'
        ].unique(), 'chromatography_stage_order']))

    # Vectorized integration per run
```

```
runs = df_clean['run'].unique()
run_stats = {}

for r in runs:
    run_data = df_clean[df_clean['run'] == r]
    run_data = run_data.sort_values('volume_ml')

    n_points = len(run_data)
    if n_points < 2:
        continue

    # Baseline calculation logic: Check for negative volume or very low volume
        to identify 'pre-column region'
    # The plan implies using negative volume as the pre-column region.
    # We filter for volume_ml <= 0 (or very close to 0 if negative values are
        missing)
    pre_column_mask = run_data['volume_ml'] <= 0
    baseline_segment = run_data[pre_column_mask]

    if len(baseline_segment) < 2:
        # Fallback if no negative volume exists, use first few points
        baseline_segment = run_data.iloc[:min(50, len(run_data))]

    if len(baseline_segment) < 2:
        baseline_uv = 0.0
    else:
        baseline_uv = baseline_segment['UV_1_280_mAU'].mean()

    y = run_data['UV_1_280_mAU'].values
    x = run_data['volume_ml'].values
    dx = np.diff(x)
    dy = np.diff(y)
    areas = (y[:-1] + y[1:]) / 2 * dx
    total_area = np.sum(areas)

    if len(run_data) < 2:
        normalized_area = total_area
    elif baseline_uv == 0:
        normalized_area = total_area
    else:
        normalized_area = total_area / baseline_uv

    if len(baseline_segment) < 2:
        noise = 0.0
    else:
        noise = baseline_segment['UV_1_280_mAU'].std()

    signal = run_data['UV_1_280_mAU'].max() - baseline_uv
    if len(run_data) < 2 or noise == 0:
        snr = float('inf') if signal > 0 else 0.0
    else:
        snr = signal / noise

    run_stats[r] = {
        'run': r,
        'run_no': run_data['run_no'].iloc[0],
        'Conc_B_%': run_data['Conc_B_%'].iloc[0],
        'Integrated_Area': total_area,
        'Normalized_Integrated_Area': normalized_area,
```

```
            'SNR': snr,
            'stage_order': stage_lookup.get(r, 0)
        }

    df_metrics = pd.DataFrame(list(run_stats.values()))

    if len(df_metrics) < 2:
        print("Insufficient data for correlation analysis.")
        return

    corr_coef = df_metrics['Normalized_Integrated_Area'].corr(df_metrics['Conc_B_%
        '])
    df_metrics['stage_order'] = df_metrics['stage_order'].fillna(0)

    stage_stats = df_metrics.groupby('stage_order').agg({
        'Normalized_Integrated_Area': 'mean',
        'Conc_B_%': 'mean'
    }).reset_index()

    if len(stage_stats) < 2:
        slope = 0.0
        trend_description = "Insufficient stages for drift analysis."
    else:
        X = stage_stats['stage_order'].values.reshape(-1, 1)
        y = stage_stats['Normalized_Integrated_Area'].values
        model = LinearRegression()
        model.fit(X, y)
        slope = model.coef_[0]
        intercept = model.intercept_

        if abs(slope) < 0.01:
            trend_description = "No significant drift detected."
        elif slope > 0:
            trend_description = f"Positive drift detected (slope: {slope:.4f})."
        else:
            trend_description = f"Negative drift detected (slope: {slope:.4f})."

    final_metrics_df = df_metrics[['run', 'run_no', 'Conc_B_%', '
        Normalized_Integrated_Area', 'SNR']].copy()
    final_metrics_df.columns = ['Run_ID', 'Run_No', 'Conc_B_%', '
        Mean_Integrated_Area', 'SNR']
    final_metrics_df['Correlation_Coefficient'] = corr_coef

    df_outliers = df_metrics[['run', 'Normalized_Integrated_Area', 'SNR', '
        stage_order']].copy()

    if len(df_outliers) < 2:
        flagged_df = pd.DataFrame(columns=['Run_ID', 'Flag Reason', 'Metric Value'
            ])
    else:
        X_out = df_outliers['stage_order'].values.reshape(-1, 1)
        y_out = df_outliers['Normalized_Integrated_Area'].values
        model_out = LinearRegression()
        model_out.fit(X_out, y_out)
        predicted = model_out.predict(X_out)
        residuals = y_out - predicted

        if len(residuals) < 2:
            residual_std = 0.0
```

```
        else:
            residual_std = np.std(residuals)

        flagged_runs = []
        for idx, row in df_outliers.iterrows():
            run_id = row['run']
            area = row['Normalized_Integrated_Area']
            snr = row['SNR']

            reasons = []

            if len(residuals) >= 2 and abs(residuals[idx]) > 2 * residual_std:
                reasons.append(f"Drift: Residual {residuals[idx]:.2f}")

            if snr < 20:
                reasons.append(f"Low SNR: {snr:.2f}")

            if len(df_outliers) > 1:
                Q1 = df_outliers['Normalized_Integrated_Area'].quantile(0.25)
                Q3 = df_outliers['Normalized_Integrated_Area'].quantile(0.75)
                IQR = Q3 - Q1
                if IQR > 0:
                    lower_bound = Q1 - 1.5 * IQR
                    upper_bound = Q3 + 1.5 * IQR
                    if area < lower_bound or area > upper_bound:
                        reasons.append(f"Area Outlier: {area:.2f}")

            if reasons:
                flagged_runs.append({
                    'Run_ID': run_id,
                    'Flag Reason': '; '.join(reasons),
                    'Metric Value': area
                })

        flagged_df = pd.DataFrame(flagged_runs)

    report_md = f"""# Chromatography QC Report

## 1. Run-Level Metrics

| Run_ID | Conc_B_% | Mean_Integrated_Area | SNR | Correlation_Coefficient |
|--------|----------|----------------------|-----|-------------------------|
"""

    for _, row in final_metrics_df.iterrows():
        report_md += f"| {row['Run_ID']} | {row['Conc_B_%']:.4f} | {row['
            Mean_Integrated_Area']:.4f} | {row['SNR']:.4f} | {row['
            Correlation_Coefficient']:.4f} |\n"

    report_md += f"""
## 2. Correlation Trends Analysis

- **Correlation Coefficient**: {corr_coef:.4f}
- **Trend Description**: {trend_description}
- **Slope**: {slope:.4f}

## 3. Flagged Runs (Drift and Outliers)

| Run_ID | Flag Reason | Metric Value |
```

```
|--------|-------------|--------------|
"""

    if len(flagged_df) > 0:
        for _, row in flagged_df.iterrows():
            report_md += f"| {row['Run_ID']} | {row['Flag Reason']} | {row['Metric
                Value']:.4f} |\n"
    else:
        report_md += "| No runs flagged. |\n"

    # Validate output structure
    expected_sections = ['1. Run-Level Metrics', '2. Correlation Trends Analysis',
        '3. Flagged Runs (Drift and Outliers)']
    for section in expected_sections:
        if section not in report_md:
            raise ValueError(f"Missing required section in report: {section}")

    output_file = os.path.join(output_dir, 'chromatography_qc_report.md')
    with open(output_file, 'w') as f:
        f.write(report_md)

    print(f"QC Report generated: {output_file}")
###
```

Listing 2: Code_3